

# SQL Notes (Organized)

---

## 1. Database Schema

---

- **Schema** = The overall database design or structure.
- It defines:
  - Tables
  - Columns
  - Data types
  - Constraints
  - Relationships

A database is usually made up of multiple **tables**.

---

## 2. SQL Basics

---

- SQL (Structured Query Language) is a non-case-sensitive language.

Example:

```
SELECT * FROM Students;
```

is the same as

```
select * from students;
```

---

## 3. Creating a Database

---

### Create a Database

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE College;
```

---

## Create Database Only If It Doesn't Exist

```
CREATE DATABASE IF NOT EXISTS database_name;
```

Example:

```
CREATE DATABASE IF NOT EXISTS College;
```

This command checks whether the database already exists. If it does, SQL will not create another one.

---

## 4. Deleting a Database

---

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE College;
```

## Delete Database Only If It Exists

```
DROP DATABASE IF EXISTS database_name;
```

Example:

```
DROP DATABASE IF EXISTS College;
```

## 5. Selecting a Database

---

Before creating tables or performing operations, select the database.

```
USE database_name;
```

Example:

```
USE College;
```

After using the `USE` command, all subsequent SQL commands will be executed on that database.

---

---

## 6. Viewing Databases

---

To display all databases available on the server:

```
SHOW DATABASES;
```

---

## 7. Creating Tables

---

Syntax:

```
CREATE TABLE table_name  
(  
    column_name datatype constraints  
);
```

Example:

```
CREATE TABLE Student  
(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

---

## 8. Deleting a Table

---

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Student;
```

---

## 9. Viewing Tables

---

To list all tables inside the selected database:

```
SHOW TABLES;
```

---

## 10. Data Types

---

### CHAR

---

- Stores strings with a **fixed length**.
- If the string is shorter than the specified size, SQL automatically pads the remaining space.

Example:

```
CHAR(10)
```

Stored value:

```
"Ram      "
```

---

### VARCHAR

---

- Stores strings with **variable length**.
- Only the actual characters are stored.
- No extra spaces are added.
- More storage efficient than `CHAR`.

Example:

```
VARCHAR(10)
```

Stored value:

```
"Ram"
```

---

## 11. Signed vs Unsigned Numbers

---

By default, integer types are **signed**.

Example:

```
TINYINT
```

Range:

-128 to 127

Using **UNSIGNED**:

TINYINT UNSIGNED

Range:

0 to 255

Unsigned numbers cannot store negative values but provide a larger positive range.

---

## 12. Constraints

---

Constraints are rules that limit or control the data that can be stored in a table.

Examples:

- PRIMARY KEY
  - NOT NULL
  - UNIQUE
  - DEFAULT
  - CHECK
  - FOREIGN KEY
- 

## 13. Primary Key

---

A **Primary Key** uniquely identifies each row in a table.

### Properties

- Cannot contain **NULL** values.
- Must be unique.
- Every table should have only one primary key (it may consist of multiple columns).

Example:

StudentID INT PRIMARY KEY

---

## 14. SQL Command Categories

---

### i) DDL (Data Definition Language)

---

Used to define or modify the database structure.

Examples:

- CREATE
  - ALTER
  - DROP
  - TRUNCATE
  - RENAME
- 

### ii) DQL (Data Query Language)

---

Used to retrieve data.

Example:

```
SELECT
```

---

### iii) DML (Data Manipulation Language)

---

Used to insert, update, or delete records.

Examples:

- INSERT
  - UPDATE
  - DELETE
- 

### iv) DCL (Data Control Language)

---

Used to control permissions.

Examples:

- GRANT
- REVOKE

---

## v) TCL (Transaction Control Language)

---

Used to manage transactions.

Examples:

- COMMIT
- ROLLBACK
- SAVEPOINT

---

## 15. Viewing All Data in a Table

---

To display every row and every column:

```
SELECT * FROM table_name;
```

Example:

```
SELECT * FROM Student;
```

---

## 16. Inserting Data

---

### Insert Specific Columns

---

Syntax:

```
INSERT INTO table_name  
(column1, column2, column3)  
VALUES  
(value1, value2, value3);
```

Example:

```
INSERT INTO Student  
(StudentID, Name, Age)  
VALUES  
(1, 'Rahul', 20);
```

---

# Insert Multiple Rows

---

Syntax:

```
INSERT INTO table_name
(column1, column2)
VALUES
(value1, value2),
(value3, value4),
(value5, value6);
```

Example:

```
INSERT INTO Student
(StudentID, Name)
VALUES
(1, 'Rahul'),
(2, 'Aman'),
(3, 'Priya');
```

Result:

| StudentID | Name  |
|-----------|-------|
| 1         | Rahul |
| 2         | Aman  |
| 3         | Priya |

---

## Important Rule

The order of values **must match** the order of columns.

Correct:

```
INSERT INTO Student
(StudentID, Name)
VALUES
(1, 'Rahul');
```

Incorrect:

```
INSERT INTO Student
(StudentID, Name)
VALUES
('Rahul', 1);
```

---



# Insert Values into All Columns

If values are provided for every column in the correct order, column names can be omitted.

Syntax:

```
INSERT INTO table_name  
VALUES  
(value1, value2, value3);
```

Example:

```
INSERT INTO Student  
VALUES  
(1, 'Rahul', 20);
```

## Quick Summary

| Command          | Purpose                        |
|------------------|--------------------------------|
| CREATE DATABASE  | Create a database              |
| DROP DATABASE    | Delete a database              |
| USE              | Select a database              |
| SHOW DATABASES   | Display all databases          |
| CREATE TABLE     | Create a table                 |
| DROP TABLE       | Delete a table                 |
| SHOW TABLES      | Display all tables             |
| SELECT *         | View all records               |
| INSERT INTO      | Insert new records             |
| PRIMARY KEY      | Unique and NOT NULL identifier |
| CHAR             | Fixed-length string            |
| VARCHAR          | Variable-length string         |
| TINYINT          | Integer (-128 to 127)          |
| TINYINT UNSIGNED | Integer (0 to 255)             |

## Part -2

---

# SQL Keys and Constraints

---

## 1. Primary Key

---

A **Primary Key** is a column (or a combination of columns) that uniquely identifies each row in a table.

### Properties

- Must contain **unique** values.
- Cannot contain **NULL** values.
- A table can have **only one Primary Key**.
- The Primary Key can consist of **one or more columns** (composite key).

### Method 1: Define Primary Key with the Column

```
CREATE TABLE table_name (  
    col_name datatype PRIMARY KEY  
);
```

### Method 2: Define Primary Key Separately

```
CREATE TABLE table_name (  
    col_name1 datatype,  
    col_name2 datatype,  
    col_name3 datatype,  
    PRIMARY KEY (col_name1)  
);
```

---

## 2. Foreign Key

---

A **Foreign Key** is a column that refers to the **Primary Key** of another table.

### Properties

- Creates a relationship between two tables.
- Can contain **duplicate (non-unique)** values.
- Can contain **NULL** values (unless specified as `NOT NULL`).
- A table can have **multiple Foreign Keys**.

## Syntax

```
CREATE TABLE table_name (  
  col_name datatype,  
  foreign_col datatype,  
  FOREIGN KEY (foreign_col)  
    REFERENCES old_table(primary_key_column)  
);
```

---

## SQL Constraints

Constraints are rules applied to table columns to maintain data integrity.

### Types of Constraints

---

#### i) NOT NULL

Ensures that a column cannot have a **NULL** value.

```
CREATE TABLE table_name (  
  col_name datatype NOT NULL  
);
```

---

#### ii) UNIQUE

Ensures that all values in a column are different.

```
CREATE TABLE table_name (  
  col_name datatype UNIQUE  
);
```

---

#### iii) PRIMARY KEY

Ensures that a column is both **UNIQUE** and **NOT NULL**.

```
CREATE TABLE table_name (  
  col_name datatype PRIMARY KEY  
);
```

or

```
CREATE TABLE table_name (  
    col_name1 datatype,  
    col_name2 datatype,  
    PRIMARY KEY (col_name1)  
);
```

---

## iv) FOREIGN KEY

Links one table to another using the Primary Key.

```
CREATE TABLE table_name (  
    col_name datatype,  
    foreign_col datatype,  
    FOREIGN KEY (foreign_col)  
        REFERENCES old_table(primary_key_column)  
);
```

---

## v) DEFAULT

Assigns a default value if no value is provided during insertion.

```
CREATE TABLE table_name (  
    col_name1 datatype DEFAULT value,  
    col_name2 datatype  
);
```

Example:

```
INSERT INTO table_name (col_name2)  
VALUES ('Value');
```

In this case, `col_name1` automatically receives its default value.

---

## vi) CHECK

A **CHECK** constraint restricts the values that can be inserted or updated in a table.

If the condition is false, the database rejects the row.

```
CREATE TABLE table_name (  
    column_name data_type,  
    CHECK (condition)  
);
```

Example:

```
CREATE TABLE Student (  
    Age INT,  
    CHECK (Age >= 18)  
);
```

Only students with age **18 or above** can be inserted.

## SELECT Statement

The **SELECT** statement is used to retrieve data from a table.

### Select All Columns

```
SELECT * FROM table_name;
```

### Select Specific Columns

```
SELECT column1, column2  
FROM table_name;
```

## Quick Summary

| Concept     | Description                                                         |
|-------------|---------------------------------------------------------------------|
| Primary Key | Uniquely identifies each row; cannot be NULL; only one per table.   |
| Foreign Key | Refers to the Primary Key of another table; duplicates are allowed. |
| NOT NULL    | Prevents NULL values.                                               |
| UNIQUE      | Prevents duplicate values.                                          |
| PRIMARY KEY | Combination of <b>UNIQUE</b> and <b>NOT NULL</b> .                  |
| FOREIGN KEY | Creates relationships between tables.                               |
| DEFAULT     | Assigns a default value if none is provided.                        |
| CHECK       | Restricts values based on a specified condition.                    |
| SELECT      | Retrieves data from a table.                                        |

# SQL Clauses, Operators, and Aggregate Functions

## 1. WHERE Clause

The `WHERE` clause is used to filter records based on a condition. Only the rows that satisfy the condition are displayed.

### Syntax

```
SELECT column_name  
FROM table_name  
WHERE condition;
```

### Example

```
SELECT marks  
FROM student  
WHERE city = 'Dhaka';
```

### Explanation

- `SELECT marks` → Displays only the **marks** column.
- `WHERE city = 'Dhaka'` → Selects only the students whose city is **Dhaka**.

Example Table:

| Name  | Marks | City       |
|-------|-------|------------|
| Rahim | 85    | Dhaka      |
| Karim | 78    | Chittagong |
| Rina  | 92    | Dhaka      |

Query:

```
SELECT marks  
FROM student  
WHERE city = 'Dhaka';
```

Output:

| Marks |
|-------|
| 85    |
| 92    |

## 2. DISTINCT

---

The `DISTINCT` keyword removes duplicate values from the result.

### Syntax

```
SELECT DISTINCT column_name  
FROM table_name;
```

### Example

```
SELECT DISTINCT city  
FROM student;
```

If the table contains:

| City     |
|----------|
| Dhaka    |
| Dhaka    |
| Rajshahi |
| Khulna   |
| Rajshahi |

Output:

| City     |
|----------|
| Dhaka    |
| Rajshahi |
| Khulna   |

---

## 3. Multiple Conditions

---

Multiple conditions can be used in the `WHERE` clause with logical operators.

Example:

```
SELECT *  
FROM student  
WHERE city = 'Dhaka'  
AND marks > 80;
```

---

## 4. Operators Used in SQL

---

Operators are commonly used inside the `WHERE` clause.

### i) Arithmetic Operators

---

Used for mathematical calculations.

| Operator | Meaning             |
|----------|---------------------|
| +        | Addition            |
| -        | Subtraction         |
| *        | Multiplication      |
| /        | Division            |
| %        | Modulus (Remainder) |

Example:

```
SELECT marks + 5  
FROM student;
```

---

### ii) Comparison Operators

---

Used to compare values.

| Operator | Meaning                  |
|----------|--------------------------|
| =        | Equal to                 |
| >        | Greater than             |
| <        | Less than                |
| >=       | Greater than or equal to |
| <=       | Less than or equal to    |
| <> or != | Not equal to             |

Example:



```
SELECT *  
FROM student  
WHERE marks >= 80;
```

---

## iii) Logical Operators

---

### AND

Both conditions must be **true**.

```
SELECT *  
FROM student  
WHERE city = 'Dhaka'  
AND marks > 80;
```

### OR

At least one condition must be **true**.

```
SELECT *  
FROM student  
WHERE city = 'Dhaka'  
OR city = 'Khulna';
```

### NOT

Reverses the condition.

```
SELECT *  
FROM student  
WHERE NOT city = 'Dhaka';
```

Shows all students **except** those from Dhaka.

---

## iv) Bitwise Operators

---

Used to perform bit-level operations.

Common bitwise operators:

| Operator | Meaning     |
|----------|-------------|
| &        | Bitwise AND |
|          | Bitwise OR  |
| ^        | Bitwise XOR |

Example:

```
SELECT 5 & 3;
```

Output:

```
1
```

---

## 5. BETWEEN Operator

The `BETWEEN` operator is used to select values within a range (inclusive).

### Syntax

```
SELECT *  
FROM table_name  
WHERE column_name BETWEEN value1 AND value2;
```

### Example

```
SELECT *  
FROM student  
WHERE marks BETWEEN 70 AND 90;
```

This returns students whose marks are between **70 and 90**, including both 70 and 90.

---

## 6. IN Operator

The `IN` operator is used to match a value from a list.

### Syntax

```
SELECT *  
FROM table_name  
WHERE column_name IN (value1, value2, value3);
```

## Example

```
SELECT *  
FROM student  
WHERE city IN ('Dhaka', 'Khulna', 'Rajshahi');
```

This returns students from Dhaka, Khulna, or Rajshahi.

---

## 7. LIMIT Clause

The `LIMIT` clause is used to display only a specified number of rows.

### Syntax

```
SELECT *  
FROM table_name  
LIMIT number;
```

## Example

```
SELECT *  
FROM student  
LIMIT 5;
```

Displays only the first 5 rows.

---

## 8. ORDER BY Clause

The `ORDER BY` clause is used to sort the output.

### Ascending Order (ASC)

```
SELECT *  
FROM student  
ORDER BY marks ASC;
```

or simply

```
SELECT *  
FROM student  
ORDER BY marks;
```

Sorts marks from lowest to highest.

---

## Descending Order (DESC)

---

```
SELECT *  
FROM student  
ORDER BY marks DESC;
```

Sorts marks from **highest to lowest**.

---

## 9. Aggregate Functions

---

Aggregate functions perform calculations on multiple rows and return a single value.

### i) COUNT()

---

Counts the number of rows.

```
SELECT COUNT(*)  
FROM student;
```

Count students from Dhaka:

```
SELECT COUNT(*)  
FROM student  
WHERE city = 'Dhaka';
```

---

### ii) MAX()

---

Returns the highest value.

```
SELECT MAX(marks)  
FROM student;
```

---

### iii) SUM()

---

Returns the total sum.

```
SELECT SUM(marks)  
FROM student;
```

---

## iv) AVG()

---

Returns the average value.

```
SELECT AVG(marks)
FROM student;
```

---

## General Syntax for Aggregate Functions

---

```
SELECT FunctionName(column_name)
FROM table_name;
```

Examples:

```
SELECT COUNT(*)
FROM student;

SELECT MAX(marks)
FROM student;

SELECT SUM(marks)
FROM student;

SELECT AVG(marks)
FROM student;
```

---

# Quick Summary

| SQL Keyword   | Purpose                             | Example                       |
|---------------|-------------------------------------|-------------------------------|
| WHERE         | Filter rows                         | WHERE city='Dhaka'            |
| DISTINCT      | Remove duplicates                   | SELECT DISTINCT city          |
| AND           | Both conditions must be true        | marks > 80 AND city='Dhaka'   |
| OR            | At least one condition must be true | city='Dhaka' OR city='Khulna' |
| NOT           | Reverse a condition                 | NOT city='Dhaka'              |
| BETWEEN       | Select values within a range        | marks BETWEEN 70 AND 90       |
| IN            | Match values from a list            | city IN ('Dhaka', 'Khulna')   |
| LIMIT         | Limit the number of rows            | LIMIT 5                       |
| ORDER BY ASC  | Sort in ascending order             | ORDER BY marks ASC            |
| ORDER BY DESC | Sort in descending order            | ORDER BY marks DESC           |
| COUNT()       | Count rows                          | COUNT(*)                      |
| MAX()         | Find the highest value              | MAX(marks)                    |
| SUM()         | Calculate the total                 | SUM(marks)                    |
| AVG()         | Calculate the average               | AVG(marks)                    |

## SQL GROUP BY

### What is GROUP BY?

`GROUP BY` collects rows that have the same value in one or more columns and returns them as a single group.

It is mostly used with **aggregate functions** such as:

- `COUNT()`
- `SUM()`
- `AVG()`
- `MAX()`
- `MIN()`

## Example Table: Student

| id | name  | city     | marks |
|----|-------|----------|-------|
| 1  | Rahim | Dhaka    | 80    |
| 2  | Karim | Dhaka    | 90    |
| 3  | Sakib | Rajshahi | 70    |
| 4  | Mina  | Dhaka    | 85    |
| 5  | Rita  | Rajshahi | 95    |

## Example 1: Count students in each city

```
SELECT city, COUNT(id)
FROM student
GROUP BY city;
```

### Output

| city     | COUNT(id) |
|----------|-----------|
| Dhaka    | 3         |
| Rajshahi | 2         |

### Explanation

- Dhaka has 3 students.
- Rajshahi has 2 students.

## Example 2: Average marks of each city

```
SELECT city, AVG(marks)
FROM student
GROUP BY city;
```

### Output

| city     | AVG(marks) |
|----------|------------|
| Dhaka    | 85         |
| Rajshahi | 82.5       |

## Important Rule

---

Every column written in `SELECT` without an aggregate function **must** appear in the `GROUP BY` clause.

### Correct

```
SELECT city, COUNT(id)
FROM student
GROUP BY city;
```

### Wrong

```
SELECT city, name, COUNT(id)
FROM student
GROUP BY city;
```

This produces an error because `name` is neither grouped nor aggregated.

---

## HAVING Clause

---

### What is HAVING?

---

`HAVING` is used to filter **groups** after `GROUP BY` .

- `WHERE` filters rows.
- `HAVING` filters groups.

### Syntax

```
SELECT column_name,
       aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

---

## Example

---

Show only cities having more than 2 students.

```
SELECT city, COUNT(id)
FROM student
GROUP BY city
HAVING COUNT(id) > 2;
```



## Output

| city  | COUNT(id) |
|-------|-----------|
| Dhaka | 3         |

Rajshahi is not shown because it has only 2 students.

---

## WHERE vs HAVING

---

| WHERE                                     | HAVING                      |
|-------------------------------------------|-----------------------------|
| Filters rows                              | Filters groups              |
| Used before GROUP BY                      | Used after GROUP BY         |
| Cannot use aggregate functions (normally) | Can use aggregate functions |

### Example of WHERE

```
SELECT *  
FROM student  
WHERE marks > 80;
```

### Example of HAVING

```
SELECT city, AVG(marks)  
FROM student  
GROUP BY city  
HAVING AVG(marks) > 80;
```

---

## General Order of SQL Clauses

---

```
SELECT column_name  
FROM table_name  
WHERE condition  
GROUP BY column_name  
HAVING condition  
ORDER BY column_name ASC;
```

## Order to Remember

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```

# UPDATE

## What is UPDATE?

**UPDATE** changes existing data in a table.

## Syntax

```
UPDATE table_name  
SET column1 = value1,  
    column2 = value2  
WHERE condition;
```

## Example

Before

| id | name  | marks |
|----|-------|-------|
| 1  | Rahim | 80    |

Increase Rahim's marks to 90.

```
UPDATE student  
SET marks = 90  
WHERE id = 1;
```

After

| id | name  | marks |
|----|-------|-------|
| 1  | Rahim | 90    |

# SQL Safe Update Mode

---

If MySQL shows:

```
Error Code: 1175
```

Disable Safe Update mode.

```
SET SQL_SAFE_UPDATES = 0;
```

---

## DELETE

### What is DELETE?

---

Deletes rows from a table.

#### Syntax

```
DELETE FROM table_name  
WHERE condition;
```

---

### Example

Delete the student whose ID is 3.

```
DELETE FROM student  
WHERE id = 3;
```

---

### Delete All Rows

```
DELETE FROM student;
```

The table remains, but all data is removed.

---

# Parent Table and Child Table

---

## Parent Table

---

The table that contains the **Primary Key**.

Example:

### Department

| dept_id | dept_name |
|---------|-----------|
| 101     | CSE       |
| 102     | EEE       |

Here, `dept_id` is the Primary Key.

---

## Child Table

---

The table that contains the **Foreign Key**.

### Student

| id | name  | dept_id |
|----|-------|---------|
| 1  | Rahim | 101     |
| 2  | Karim | 102     |

Here, `dept_id` is a Foreign Key that references the `Department` table.

---

## Cascading in Foreign Keys

---

Cascading automatically reflects changes made in the parent table to the child table.

## ON DELETE CASCADE

---

If a row is deleted from the parent table, the related rows in the child table are automatically deleted.

## Example

Department

| dept_id |
|---------|
| 101     |

Student

| id | dept_id |
|----|---------|
| 1  | 101     |
| 2  | 101     |

```
DELETE FROM department
WHERE dept_id = 101;
```

Both student records are automatically deleted.

---

## ON UPDATE CASCADE

If the Primary Key changes in the parent table, the Foreign Key in the child table is automatically updated.

## Example

```
UPDATE department
SET dept_id = 201
WHERE dept_id = 101;
```

The child table's `dept_id` also changes from `101` to `201`.

---

## ALTER TABLE

ALTER is used to change the structure (schema/design) of an existing table.

### 1. Add a Column

#### General Syntax

```
ALTER TABLE table_name
ADD COLUMN column_name datatype constraints;
```

## Example

```
ALTER TABLE Student  
ADD COLUMN Email VARCHAR(100);
```

---

## 2. Remove (Drop) a Column

### General Syntax

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

## Example

```
ALTER TABLE Student  
DROP COLUMN Email;
```

---

## 3. Rename a Table

### General Syntax

```
ALTER TABLE table_name  
RENAME TO new_table_name;
```

## Example

```
ALTER TABLE Student  
RENAME TO Students;
```

---

## 4. Change (Rename) a Column Name

**Note:** The syntax depends on the database.

## MySQL

### General Syntax

```
ALTER TABLE table_name  
CHANGE COLUMN old_column_name new_column_name new_datatype constraints;
```

## Example

```
ALTER TABLE Student  
CHANGE COLUMN Name Full_Name VARCHAR(100);
```

---

## 5. Modify a Column (Change Data Type or Constraints)

---

### MySQL

#### General Syntax

```
ALTER TABLE table_name  
MODIFY COLUMN column_name new_datatype constraints;
```

## Example

```
ALTER TABLE Student  
MODIFY COLUMN Age INT NOT NULL;
```

---

## TRUNCATE TABLE

---

TRUNCATE deletes all rows from a table but keeps the table structure.

#### General Syntax

```
TRUNCATE TABLE table_name;
```

## Example

```
TRUNCATE TABLE Student;
```

#### Result:

- Deletes all records from the table.
  - Keeps the table structure.
  - Cannot be rolled back in many database systems.
-

# DROP TABLE

DROP TABLE deletes the entire table, including its structure and data.

## General Syntax

```
DROP TABLE table_name;
```

## Example

```
DROP TABLE Student;
```

### Result:

- Deletes the table permanently.
- Removes both the table structure and all data.

# Summary

| Command                    | Purpose                            | General Syntax                                                                            |
|----------------------------|------------------------------------|-------------------------------------------------------------------------------------------|
| Add Column                 | Add a new column                   | <code>ALTER TABLE table_name ADD COLUMN column_name datatype constraints;</code>          |
| Drop Column                | Remove an existing column          | <code>ALTER TABLE table_name DROP COLUMN column_name;</code>                              |
| Rename Table               | Change the table name              | <code>ALTER TABLE table_name RENAME TO new_table_name;</code>                             |
| Change Column Name (MySQL) | Rename a column                    | <code>ALTER TABLE table_name CHANGE COLUMN old_name new_name datatype constraints;</code> |
| Modify Column (MySQL)      | Change data type or constraints    | <code>ALTER TABLE table_name MODIFY COLUMN column_name datatype constraints;</code>       |
| Truncate Table             | Delete all rows but keep the table | <code>TRUNCATE TABLE table_name;</code>                                                   |
| Drop Table                 | Delete the table and its data      | <code>DROP TABLE table_name;</code>                                                       |

# TRUNCATE

Deletes **all rows** from a table quickly.



```
TRUNCATE TABLE student;
```

## Characteristics

- Deletes all rows.
- Cannot use a `WHERE` clause.
- Faster than `DELETE`.
- Table structure remains.

## DROP TABLE

Deletes the **entire table**, including:

- All rows
- All columns
- Table structure

```
DROP TABLE student;
```

After executing this command, the `student` table no longer exists.

## DELETE vs TRUNCATE vs DROP

| Feature                  | DELETE | TRUNCATE | DROP                 |
|--------------------------|--------|----------|----------------------|
| Deletes rows             | ✓ Yes  | ✓ Yes    | ✗ No (Deletes table) |
| Deletes table structure  | ✗ No   | ✗ No     | ✓ Yes                |
| Can use WHERE            | ✓ Yes  | ✗ No     | ✗ No                 |
| Can delete selected rows | ✓ Yes  | ✗ No     | ✗ No                 |
| Table remains            | ✓ Yes  | ✓ Yes    | ✗ No                 |
| Usually faster           | ✗      | ✓        | ✓                    |

# Quick Revision

---

- **GROUP BY** → Groups rows with the same value.
- **Aggregate Functions** → `COUNT()` , `SUM()` , `AVG()` , `MAX()` , `MIN()` .
- **HAVING** → Filters grouped data.
- **WHERE** → Filters individual rows.
- **UPDATE** → Changes existing values.
- **DELETE** → Removes selected rows.
- **TRUNCATE** → Removes all rows but keeps the table.
- **DROP** → Removes the entire table.
- **ALTER** → Changes the table structure.
- **ON DELETE CASCADE** → Deletes related child records automatically.
- **ON UPDATE CASCADE** → Updates related child foreign keys automatically.

## SQL Joins, UNION, Subqueries, Views, and Aliases (Complete Notes)

---

### 1. SQL JOIN

---

A **JOIN** is used to combine rows from **two or more tables** based on a related column.

**Note:** A Foreign Key is not required to perform a JOIN. You only need a matching condition in the `ON` clause.

### General Syntax

---

```
SELECT column_name(s)
FROM TableA
JOIN TableB
ON TableA.column_name = TableB.column_name;
```

---

# Example Tables

## Students

| StudentID | Name    |
|-----------|---------|
| 1         | Alice   |
| 2         | Bob     |
| 3         | Charlie |
| 4         | David   |

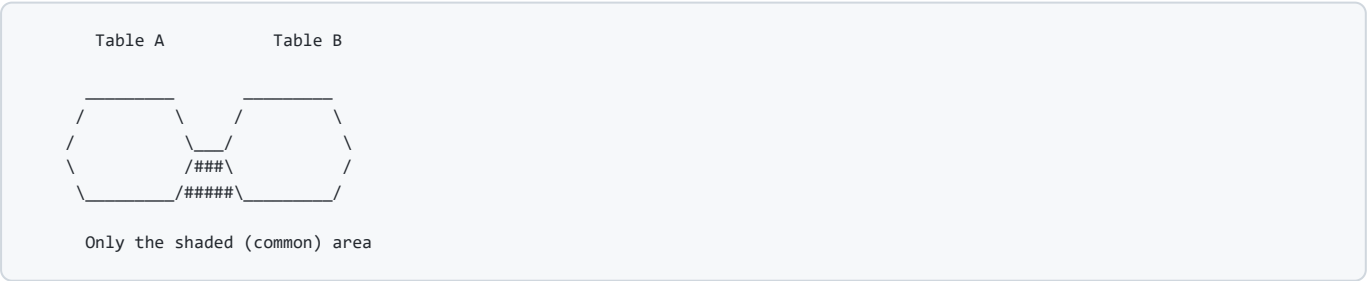
## Marks

| StudentID | Marks |
|-----------|-------|
| 2         | 80    |
| 3         | 90    |
| 4         | 75    |
| 5         | 60    |

## i) INNER JOIN

Returns only the matching records that exist in both tables.

## Venn Diagram



## General Syntax

```
SELECT column(s)
FROM TableA
INNER JOIN TableB
ON TableA.column_name = TableB.column_name;
```

## Example

```
SELECT Students.StudentID, Name, Marks
FROM Students
INNER JOIN Marks
ON Students.StudentID = Marks.StudentID;
```

## Output

| StudentID | Name    | Marks |
|-----------|---------|-------|
| 2         | Bob     | 80    |
| 3         | Charlie | 90    |
| 4         | David   | 75    |

## ii) LEFT JOIN

Returns **all records from the left table** and matching records from the right table.

If there is no match, **NULL** is returned.

## Venn Diagram

Table A

Table B



## General Syntax

```
SELECT column(s)
FROM TableA
LEFT JOIN TableB
ON TableA.column_name = TableB.column_name;
```

## Example

```
SELECT Students.StudentID, Name, Marks
FROM Students
LEFT JOIN Marks
ON Students.StudentID = Marks.StudentID;
```

## Output

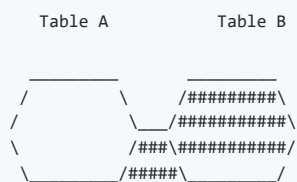
| StudentID | Name    | Marks |
|-----------|---------|-------|
| 1         | Alice   | NULL  |
| 2         | Bob     | 80    |
| 3         | Charlie | 90    |
| 4         | David   | 75    |

## iii) RIGHT JOIN

Returns **all records from the right table** and matching records from the left table.

If there is no match, **NULL** is returned.

## Venn Diagram



## General Syntax

```
SELECT column(s)
FROM TableA
RIGHT JOIN TableB
ON TableA.column_name = TableB.column_name;
```

## Example

```
SELECT Students.StudentID, Name, Marks
FROM Students
RIGHT JOIN Marks
ON Students.StudentID = Marks.StudentID;
```

## Output

| StudentID | Name    | Marks |
|-----------|---------|-------|
| 2         | Bob     | 80    |
| 3         | Charlie | 90    |
| 4         | David   | 75    |
| NULL      | NULL    | 60    |

## iv) FULL OUTER JOIN

Returns:

- All matching records
- All rows from Table A
- All rows from Table B

MySQL does not support FULL OUTER JOIN directly.

## Venn Diagram

Table A      Table B



Entire both circles

## Standard SQL Syntax

```
SELECT column(s)
FROM TableA
FULL OUTER JOIN TableB
ON TableA.column_name = TableB.column_name;
```

# MySQL Alternative

```
SELECT *
FROM TableA
LEFT JOIN TableB
ON TableA.column_name = TableB.column_name

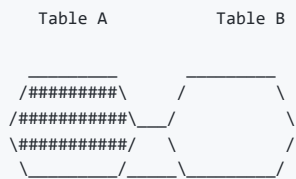
UNION

SELECT *
FROM TableA
RIGHT JOIN TableB
ON TableA.column_name = TableB.column_name;
```

## v) LEFT EXCLUSIVE JOIN

Returns only the records that exist in **Table A** but not in **Table B**.

### Venn Diagram



### General Syntax

```
SELECT column(s)
FROM TableA
LEFT JOIN TableB
ON TableA.column_name = TableB.column_name
WHERE TableB.column_name IS NULL;
```

### Example

```
SELECT Name
FROM Students
LEFT JOIN Marks
ON Students.StudentID = Marks.StudentID
WHERE Marks.StudentID IS NULL;
```

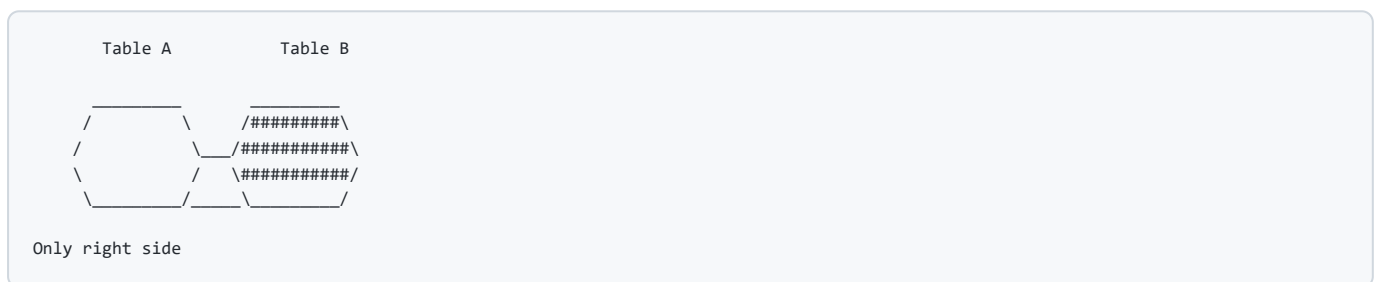
## Output

| Name  |
|-------|
| Alice |

## vi) RIGHT EXCLUSIVE JOIN

Returns only the records that exist in **Table B** but not in **Table A**.

## Venn Diagram



## General Syntax

```
SELECT column(s)
FROM TableA
RIGHT JOIN TableB
ON TableA.column_name = TableB.column_name
WHERE TableA.column_name IS NULL;
```

## Example

```
SELECT Marks.StudentID
FROM Students
RIGHT JOIN Marks
ON Students.StudentID = Marks.StudentID
WHERE Students.StudentID IS NULL;
```

## Output

| StudentID |
|-----------|
| 5         |



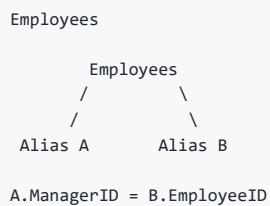
## vii) SELF JOIN

A **Self Join** joins a table with itself using aliases.

It is commonly used for:

- Employee–Manager relationships
- Parent–Child relationships
- Comparing rows within the same table

### Diagram



### General Syntax

```
SELECT column(s)
FROM TableName AS A
JOIN TableName AS B
ON A.column_name = B.column_name;
```

### Example Table

| EmployeeID | Employee | ManagerID |
|------------|----------|-----------|
| 1          | John     | NULL      |
| 2          | Alice    | 1         |
| 3          | Bob      | 1         |

### Example

```
SELECT
A.Employee,
B.Employee AS Manager
FROM Employees AS A
JOIN Employees AS B
ON A.ManagerID = B.EmployeeID;
```

## Output

| Employee | Manager |
|----------|---------|
| Alice    | John    |
| Bob      | John    |

---

## 2. SQL Alias

An **Alias** is a temporary name given to a table or a column.

It improves query readability.

### Table Alias

---

#### General Syntax

```
FROM table_name AS alias_name;
```

or

```
FROM table_name alias_name;
```

#### Example

```
SELECT S.Name  
FROM Students AS S;
```

---

## Column Alias

#### General Syntax

```
SELECT column_name AS alias_name  
FROM table_name;
```

#### Example

```
SELECT Name AS Student_Name  
FROM Students;
```

---

## 3. UNION

---

Used to combine the results of **two or more SELECT statements**.

It removes duplicate rows.

### Diagram

---

```
SELECT 1
  \
   \ UNION
  /
SELECT 2
  ↓
One combined result
(No duplicates)
```

### Conditions

---

1. Every SELECT statement must return the same number of columns.
2. Columns must have similar data types.
3. Columns must appear in the same order.

### General Syntax

---

```
SELECT column(s)
FROM TableA

UNION

SELECT column(s)
FROM TableB;
```

### Example

---

```
SELECT Name
FROM Students

UNION

SELECT Name
FROM Teachers;
```

---

# UNION ALL

---

Returns **all records**, including duplicate rows.

## General Syntax

---

```
SELECT column(s)
FROM TableA

UNION ALL

SELECT column(s)
FROM TableB;
```

## Difference

---

| UNION              | UNION ALL        |
|--------------------|------------------|
| Removes duplicates | Keeps duplicates |
| Slightly slower    | Usually faster   |

---

## 4. SQL Subqueries

---

A **Subquery** is a query written inside another SQL query.

A subquery can be used in:

- WHERE
- FROM
- SELECT

## Diagram

---

```
Outer Query

SELECT ...

WHERE column =
(
    SELECT ...
)
```

## A) Subquery in WHERE

---

### General Syntax

```
SELECT column(s)
FROM table_name
WHERE column_name operator
(
    SELECT column_name
    FROM table_name
);
```

### Example

```
SELECT Name
FROM Students
WHERE StudentID =
(
    SELECT StudentID
    FROM Marks
    WHERE Marks = 90
);
```

---

## B) Subquery in FROM

---

### General Syntax

```
SELECT column(s)
FROM
(
    SELECT ...
) AS TempTable;
```

### Example

```
SELECT AVG(Marks)
FROM
(
    SELECT Marks
    FROM Marks
) AS TempMarks;
```

---

## C) Subquery in SELECT

---

### General Syntax

```
SELECT
column_name,
(
    SELECT function(column_name)
    FROM another_table
)
FROM table_name;
```

### Example

```
SELECT
Name,
(
    SELECT AVG(Marks)
    FROM Marks
) AS AverageMarks
FROM Students;
```

---

## 5. MySQL Views

---

A **View** is a virtual table created from the result of a `SELECT` statement.

A view stores the SQL query, **not the actual data**.

### Diagram

---

```
graph TD
    A[Base Tables] --> B[SELECT Query]
    B --> C[VIEW]
    C --> D[User]
```

### Advantages

---

- Simplifies complex queries
  - Improves security
  - Reusable
  - Acts like a virtual table
-

# Create View

---

## General Syntax

```
CREATE VIEW view_name AS  
SELECT column(s)  
FROM table_name;
```

## Example

```
CREATE VIEW StudentMarks AS  
SELECT Name, Marks  
FROM Students  
INNER JOIN Marks  
ON Students.StudentID = Marks.StudentID;
```

---

## View Data

```
SELECT *  
FROM StudentMarks;
```

---

## Update View

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column(s)  
FROM table_name;
```

---

## Delete View

```
DROP VIEW view_name;
```

---

# Quick Summary

| Topic                | Purpose                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------|
| INNER JOIN           | Returns only matching rows from both tables                                             |
| LEFT JOIN            | Returns all rows from the left table and matching rows from the right table             |
| RIGHT JOIN           | Returns all rows from the right table and matching rows from the left table             |
| FULL OUTER JOIN      | Returns all rows from both tables (MySQL uses <code>LEFT JOIN UNION RIGHT JOIN</code> ) |
| LEFT EXCLUSIVE JOIN  | Returns only the rows in the left table that have no match                              |
| RIGHT EXCLUSIVE JOIN | Returns only the rows in the right table that have no match                             |
| SELF JOIN            | Joins a table with itself using aliases                                                 |
| ALIAS                | Temporary name for a table or column                                                    |
| UNION                | Combines results and removes duplicates                                                 |
| UNION ALL            | Combines results and keeps duplicates                                                   |
| SUBQUERY             | A query inside another SQL query                                                        |
| VIEW                 | A virtual table created from a SELECT statement                                         |